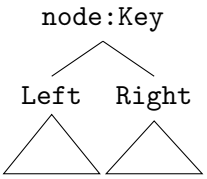
3-16. ARTIFICIAL INTELLIGENCE

1. A Binary Search Tree (BST) is a particular type of dynamic data structure that uses a key for search and storage of data items. These keys are kept in sorted order.

   Each node of a BST stores a *unique* key (e.g. an integer) and (optionally) a value associated with that key; and has two edges pointing away from it, one for the left sub-tree, and one for the right sub-tree:



   The invariant property of a BST is in-order traversal, meaning that:

   • The key stored in each node must be greater than or equal to any key stored in its `Left` sub-tree;

   • The key stored in each node must be less than or equal to any key stored in its `Right` sub-tree;

   i.e., $\forall k \in \mathit{Left}.k < \mathit{Key} \wedge \forall k \in \mathit{Right}.\mathit{Key} < k$ (i.e. no duplicates).

   This question explores how Prolog can be used to represent a BST and perform specific operations on it.

   a) Define a Prolog representation for an empty, and a non-empty BST.

   [4]

   b) Define a predicate `lookup(+Key, +BST)` which is true if key `Key` can be found in binary search tree `BST`.

   [4]

   c) Define a predicate `insert(+Key, +BST1, ?BST2)` which returns true if binary search tree `BST2` is the same as binary search tree `BST1` but with key `Key` inserted.

   [4]

   d) Define a predicate `delete(+Key, +BST1, ?BST2)` which returns true if binary search tree `BST2` is the same as binary search tree `BST1` but with key `Key` deleted. Explain what happens if the key is not found in the tree.

   [4]

   e) Define a predicate `verify(+BT)` which succeeds if binary tree `BT` satisfies the invariant property of in-order traversal (i.e. is a BST).

   [4]

   Note: the next page has additional information on the algorithms for delete and verify.

Delete

The action of:

- Deleting a node from an empty tree returns the empty tree;
- Deleting a node with empty sub-trees returns the empty tree;
- Deleting a node with one empty sub-tree returns the non-empty sub-tree.

To delete a node with two non-empty sub-trees:

- Get the in-order successor to the node from its right sub-tree;
- Replace this node's key with the successor key;
- Delete the successor-node from the sub-tree!

To find the in-order successor, look for the leftmost node with an empty left sub-tree.

Verify

In a BST, the set of possible keys is a complete order, with a minimum value `MinKey` and a maximum value `MaxKey`.

Therefore, the key stored in every node must be greater than or equal to `MinKey` and less than or equal to `MaxKey`.

Then:

1. From the root node with key `RootKey`:
   - For every key `Key` in the `Left` subtree: $\texttt{MinKey} \leq \texttt{Key} < \texttt{RootKey}$;
   - For every key `Key` in the `Right` subtree: $\texttt{RootKey} < \texttt{Key} \leq \texttt{MaxKey}$.
2. From the root of the `Left` subtree with key `LKey`:
   - For every key `Key` in its `Left` subtree: $\texttt{MinKey} \leq \texttt{Key} < \texttt{LKey}$;
   - For every key `Key` in its `Right` subtree: $\texttt{LKey} < \texttt{Key} \leq \texttt{RootKey}$.
3. From the root of the `Right` subtree with key `RKey`...

And so on...

2. **The Missionaries and Cannibals Problem (Variation 1)**. Three devout missionaries are on one bank of a wide river with three hungry cannibals, and they all want to cross the river to get to the other bank. They are restricted, however, by the following conditions:

- None of them dare swim, for there are crocodiles in the river;
- There is only one rowing boat with one set of oars;
- At most two (2) people can ride in the boat for each crossing;
- All missionaries are, **but only one cannibal** is, able to row the boat;
- Unless there are no missionaries, there must never be fewer missionaries than cannibals on either bank, otherwise the cannibals will eat the missionaries.

a) Give a state space formulation that could be used with the General Graph Search Engine (GGSE). Specify the initial state and the goal state.

[3]

b) Define a relation (predicate) `movable(Bank, Rowers)` which is true if `Rowers` is a permissible combination of rowers given the missionaries and cannibals who are on the bank `Bank`.

[3]

c) Define a relation (predicate) `satisfiable(Bank)` which is true if `Bank` satisfies the constraints on the number of missionaries and cannibals on a bank.

[3]

d) Hence define a *single* state-change rule that could be used by the GGSE to compute all the state changes (new states) from an old state. Explain why this depends on the definition of `movable/2`.

[3]

e) Explain how the Iterative Deepening Depth-First search algorithm works.

Explain how the basic GGSE could be modified to implement this algorithm.

Evaluate, with justification, the Iterative Deepening Depth-First search algorithm with respect to appropriate criteria.

Comment on the suitability of this algorithm for solving the Missionaries and Cannibals Problem (Variation 1).

[8]

3. a) In the context of the A* search algorithm, define the terms *heuristic* and *admissible heuristic*.

Explain why admissibility is essential to the A* search algorithm.

[4]

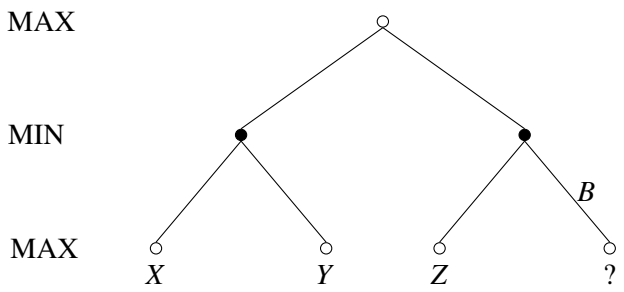
b) In the context of the A* search algorithm, discuss the criteria that might be used to choose which heuristic (or heuristics) are implemented, when there is more than one possible option.

[4]

c) Given the specification of a graph $G_S = \langle S, Op \rangle$, give an inductive definition of the paths of the graph, and explain how the A* search algorithm explores the paths defined in this way.

[4]

d) In the context of the AlphaBeta algorithm for 2-player games, consider the following tree:



Explain what happens to branch B, assuming: firstly that $X < Y$ and $X > Z$; and secondly that $X < Y$ and $X < Z$.

[4]

e) Give a data structure and algorithms for path-finding, path-following and path-learning, that could be used by a robot exploring a grid-like maze.

[4]

4. In answering this question, it is essential that any KE-trees (proofs) are properly annotated to show the inference steps (i.e. showing the inference rule used, and line number of the major and minor premises).

a) In the context of Propositional Logic, define the term *valuation* and briefly explain how a valuation can be used to give a meaning to formulas of Propositional Logic.

In the context of Predicate Logic, define the term *model* and briefly explain how a model can be used to give a meaning to predicates and formulas of Predicate Logic.

[6]

b) Consider the following English statements:

Every academic is paid-not-enough.
Anyone paid-not-enough is poor.
Every academic has too-much-work.
Anyone who has too-much-work is late-with-something.
Anyone who is poor and late-with-something is stressed.
Fred is an academic.

Formalise these statements in Predicate Logic, convert them to a set of Horn clauses, and then construct a proof, using proof-by-negation and resolution, to show that *Fred is stressed*. The proof should show all the unifiers

[6]

c) In the *Obligatio Game*, a finite number n of rounds is chosen, depending on the severity of the exam. The examiner then gives the candidate successive assertions, $\phi_1, \phi_2, \ldots, \phi_n$ that she has to either *accept* or *reject* as each one is put forward. In the former case, ϕ_i is added to the candidate's stock of commitments; in the latter case, the negation $\neg\phi_i$ is added.

The candidate passes the exam if she maintains the consistency of her stock of commitments throughout all n assertions.

Suppose the candidate is exposed (successively) to the following three ($n = 3$) assertions:

1. $q \rightarrow r$
2. $(p \wedge q) \leftrightarrow (q \wedge r)$
3. p

Using the KE calculus, show what sequences of answers would ensure that the student passes the exam.

(Hint: use KE for model building, for all combinations of *accept* or *reject* of the assertions. If the proof has at least one open branch, then this implies a consistent sequence of answers.)

[8]

5. The **Farmer-Wolf-Goat-Cabbage** River Crossing Problem (FWGC). A farmer, wolf, goat and cabbage are on one bank of a wide river with a boat, and they all want to get to the other bank. The farmer can transport one of the wolf, goat or cabbage in the boat across the river, or he can go by himself. However, if he leaves the wolf with the goat, the wolf will eat the goat; if he leaves the goat with the cabbage, the goat will eat the cabbage. How do they all get across?

Let f stand for the proposition that "the farmer is on the left bank", so that $\neg f$ stands for the proposition that "the farmer is on the right bank". Let w, g, c and their negations correspond similarly to the locations of the wolf, goat and cabbage respectively.

a) From an initial state given by $\{f, w, g, c\}$, draw the search space for the FWGC problem to depth $d = 3$.

[4]

b) From this search space, specify a Kripke Model.

[3]

c) What property does the accessibility relation of part (b) satisfy?

[2]

d) Using the calculus for Modal KE, show that the formula $P \rightarrow \Box\Diamond P$ is not valid in modal logic **K**, but is valid in modal logic **S5**.

[4]

e) In the context of the Event Calculus, define the term *fluent*.

Using boolean fluents to represent the location of the farmer, wolf, goat and cabbage, specify the actions in the FWGC problem using `initiates` axioms.

Briefly explain, with an example, how a *narrative* is represented in the Event Calculus, and how a narrative could be used to solve the FWGC problem.

[7]